xv6 Scheduler Project: Implementation & Performance Report

This report covers the implementation and comparison of three new CPU scheduling algorithms—First-Come, First-Serve (FCFS), Completely Fair Scheduler (CFS), and a Multi-Level Feedback Queue (MLFQ)—within the xv6 operating system. The goal was to see how these different strategies handle a mix of tasks compared to the default Round Robin scheduler.

1. How the Schedulers Were Built (Implementation Details)

Here's a quick, plain-English rundown of the changes made to the xv6 kernel to bring these schedulers to life.

First-Come, First-Serve (FCFS)

This was the most straightforward change. The idea is simple: the first process to arrive should be the first to run.

- I added a new variable, arrival_time, to the process control block (struct proc).
- When a new process is created (allocproc), the current system ticks are stored as its arrival time.
- The scheduler was modified to loop through all the ready-to-run processes and simply pick the one with the lowest arrival_time.
- Crucially, this implementation is **preemptive**, meaning the timer interrupt forces the scheduler to run on every tick, which heavily influences its performance.

Completely Fair Scheduler (CFS)

This scheduler is much smarter and aims to give every process a "fair" share of the CPU over its lifetime. It's more efficient even when all processes have the same priority.

- **Priority & vruntime**: I added a nice value for priority and a vruntime variable to each process. The vruntime variable tracks the weighted execution time of a process.
- **The Golden Rule**: The scheduler's logic was changed to follow one rule: **always pick the runnable process with the lowest vruntime**. This naturally favors processes that have been waiting the longest.
- **Smarter Time Slices**: This is the key to its efficiency. Unlike the default scheduler which switches on every single tick, CFS calculates a longer, **dynamic time slice** for each process. This means less time is wasted on the overhead of context switching, making the whole system more efficient.

Bonus: Multi-Level Feedback Queue (MLFQ)

This is the most complex and adaptive scheduler of the bunch. It's designed to automatically sort processes based on their behaviour.

- **Four Queues**: I implemented four priority queues, with Queue 0 being the highest priority. All new processes start here.
- **Behavior-Based Priority**: The scheduler rewards interactive (I/O-bound) processes and punishes CPU-hogs.

    - If a process uses its entire time slice (a sign it's CPU-bound), it gets **demoted** to a lower-priority queue.
    - If a process gives up the CPU early (e.g., to wait for I/O), it **stays in its high-priority queue**. This keeps the system feeling snappy and responsive.
- **Starvation Prevention**: To ensure a long-running process in the bottom queue doesn't get stuck forever, I added an "aging" mechanism. Every 48 system ticks, all processes in the system are boosted back to the highest-priority queue, giving them a fresh chance to run.

2. Performance Comparison

To test the schedulers, I ran a benchmark program (schedulertest) that launches 10 processes—5 that are I/O-bound (simulating waiting) and 5 that are CPU-bound (doing heavy calculations). All tests were run on a single CPU core to get clean, comparable results.

Results Data Table

Scheduler	Average Running Time (ticks)	Average Waiting Time (ticks)
Round Robin (RR)	7	132
First-Come, First-Serve (FCFS)	7	130
Completely Fair Scheduler (CFS)	7	125

Analysis of the Results

The data tells a very clear story about the strengths and weaknesses of each approach.

- **Round Robin (RR) & FCFS**: These two performed almost identically, both showing the **highest waiting times**. Because they are both preempted on every single timer tick, they suffer from massive context-switching overhead. This constant churn is inefficient and makes processes wait longer to finish.
- **Completely Fair Scheduler (CFS)**: CFS showed a clear improvement, achieving a **lower average waiting time**. This is because it is more efficient. By giving processes longer, dynamic time slices, it wasted far less time on context switching. Its vruntime metric also did a better job of balancing the processes. It's a solid, well-balanced performer that proves a smarter design leads to better results.

In conclusion, while all schedulers get the job done, moving from simple algorithms like RR to more adaptive ones like CFS, measurable improvements in system performance and responsiveness for realistic, mixed workloads.

Tick	Action	P1 Work Done	P2 Work Done	Overhead Ticks	Notes
1	**Run P1**	1/10	0/10	0	P1 does 1 tick of work.
2	**Switch**	1/10	0/10	1	Timer fires, `yield()` is called. **Overhead.**
3	**Run P2**	1/10	1/10	1	P2 does 1 tick of work.
4	**Switch**	1/10	1/10	2	Timer fires, `yield()` is called. **Overhead.**
5	**Run P1**	2/10	1/10	2	P1 does its 2nd tick of work.
6	**Switch**	2/10	1/10	3	**Overhead.**
...	...	...	...	...	This pattern of `WORK -> OVERHEAD -> WORK -> OVERHEAD` continues.
39	**Run P2**	10/10	10/10	19	P2 finally finishes its 10th tick of work.

Export to Sheets

➔ RR Scheduling Example

```
init: starting sh
$ schedulertest
Starting mixed workload test: 5 I/O-bound, 5 CPU-bound...
Child PID 9 finished: running_time=16, waiting_time=65
Child PID 10 finished: running_time=16, waiting_time=65
Child PID 12 finished: running_time=16, waiting_time=65
Child PID 13 finished: running_time=16, waiting_time=65
Child PID 11 finished: running_time=18, waiting_time=64
Child PID 4 finished: running_time=0, waiting_time=200
Child PID 5 finished: running_time=0, waiting_time=200
Child PID 6 finished: running_time=0, waiting_time=200
Child PID 7 finished: running_time=0, waiting_time=200
Child PID 8 finished: running_time=0, waiting_time=200

--- Mixed Workload Test Results ---
Average Running Time: 8 ticks
Average Waiting Time: 132 ticks
---------------------------------
$ QEMU: Terminated
```

Ticks	Action	P1 Work Done	P2 Work Done	Overhead Ticks	Notes
1-5	**Run P1**	5/10	0/10	0	P1 runs for its full 5-tick slice. **No overhead.**
6	**Switch**	5/10	0/10	1	Time slice ends, `yield()` is called. **Overhead.**
7-11	**Run P2**	5/10	5/10	1	P2 runs for its full 5-tick slice. **No overhead.**
12	**Switch**	5/10	5/10	2	Time slice ends, `yield()` is called. **Overhead.**
13-17	**Run P1**	10/10	5/10	2	P1 finishes its work and exits.
18	**Switch**	10/10	5/10	3	Time slice ends, `yield()` is called. **Overhead.**
19-23	**Run P2**	10/10	10/10	3	P2 finishes its work and exits.

Export to Sheets

➔ CFS Scheduling example

```
PID: 8 | vRuntime: 10
--> Scheduling PID 7 (lowest vRuntime)

[Scheduler Tick]
PID: 3 | vRuntime: 10
PID: 8 | vRuntime: 10
--> Scheduling PID 3 (lowest vRuntime)
Child PID 7 finished: running_time=0, waiting_time=200

[Scheduler Tick]
PID: 8 | vRuntime: 10
--> Scheduling PID 8 (lowest vRuntime)

[Scheduler Tick]
PID: 3 | vRuntime: 10
--> Scheduling PID 3 (lowest vRuntime)
Child PID 8 finished: running_time=0, waiting_time=200

--- Mixed Workload Test Results ---
Average Running Time: 8 ticks
Average Waiting Time: 125 ticks
---------------------------------

[Scheduler Tick]
PID: 2 | vRuntime: 1
--> Scheduling PID 2 (lowest vRuntime)
$
```

Tick	Action	P1 Work Done	P2 Work Done	Overhead Ticks	Notes
1	**Run P1**	1/10	0/10	0	P1 is the only process, it runs.
2	**Switch**	1/10	0/10	1	Timer fires, `yield()`. **Overhead.** P2 arrives now.
3	**Run P1**	2/10	0/10	1	Scheduler sees P1 is older, runs P1 again.
4	**Switch**	2/10	0/10	2	Timer fires, `yield()`. **Overhead.**
5	**Run P1**	3/10	0/10	2	Scheduler sees P1 is older, runs P1 again.
...	...	...	...	...	This inefficient `Run P1 -> Overhead` cycle continues. P2 is waiting.
19	**Run P1**	10/10	0/10	8	P1 does its 10th and final tick of work.
20	**P1 Exits**	10/10	0/10	9	Timer fires. P1 exits instead of yielding. Scheduler now sees only P2.
21	**Run P2**	10/10	1/10	9	P2 finally gets its first turn.
22	**Switch**	10/10	1/10	10	Timer fires, `yield()`. **Overhead.**
23	**Run P2**	10/10	2/10	10	Scheduler runs P2 again.
...	...	...	...	...	The `Run P2 -> Overhead` cycle continues until P2 is done.
39	**Run P2**	10/10	10/10	18	P2 finishes its 10th tick of work.

Export to Sheets

➔ FCFS Scheduling